FLOOD GUARD 360 24/7 EVERY HOUR

INDIVIDUAL THESIS

Dhananjaya Bandara Bulumulla

IT22278258

Individual component: Real Time Drain Water Level Monitoring, Rainfall Monitoring, and Multi-Point Blockage Detection for Flood Risk Management

Group ID: R26-IT-031

Report submitted in partial fulfilment of the requirements for the BSc (Hons) Degree in Information Technology Specialization in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

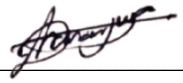
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Dhananjaya Bandara Bulumulla

Student ID: IT22278258

Signature:  Date: __10/11/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr Deemantha Siriwardana		10/11/2026

ABSTRACT

Floods in urban areas of sri lanka are often caused by heavy rainfall, poor drainage systems, and hidden blockages in drains and water channels. Traditional flood monitoring methods mainly depend on manual observation or measurements taken from a single location. Because of this, it is difficult to identify changes in water levels and water flow before flooding occurs. This research presents an iot based drainage monitoring system that continuously monitors water levels and rainfall conditions and identifies unusual water flow patterns.

The proposed system uses ESP32 based IoT devices and ultrasonic sensors to measure water levels in drains and river channels continuously. A rainfall sensor is connected to the main monitoring unit to detect rainfall conditions. For blockage detection, a separate ESP32-S3 unit uses three ultrasonic sensors placed at upstream, middle, and downstream locations. The sensors measure the distance between the sensor and the water surface, and these measurements are converted into water levels based on the installation height of each sensor. The system compares the water levels at nearby locations to identify unusual differences that continue over time. These differences may indicate a possible blockage and help estimate its location.

The sensor data are sent through Wi-Fi to a backend server using HTTP communication. The server stores the water level measurements and blockage information for continuous monitoring and further analysis. This information can also be connected to the main flood management dashboard and map-based visualization developed by the research team. The proposed system aims to reduce the need for manual inspections, identify unusual drainage conditions earlier, and help residents and relevant authorities make timely decisions. It also provides a foundation for future development of water level prediction using historical sensor data and weather information.

Keywords: IoT, drain monitoring, water level monitoring, rainfall detection, blockage detection

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my dissertation supervisor, Mr Deemantha Siriwardana, from the Faculty of Computing at the Sri Lanka Institute of Information Technology, for their valuable guidance, constructive feedback, and continuous support throughout this research project. Their advice and recommendations significantly contributed to improving both the technical development and academic quality of this research.

I would also like to thank the Department of Information Technology, Sri Lanka Institute of Information Technology (SLIIT), for providing the necessary academic environment, facilities, and resources required to conduct this research successfully.

My sincere appreciation is extended to the members of my research team for their cooperation, encouragement, and valuable contribution throughout the development of the overall flood-management system. Their discussions, technical support, and collaborative efforts helped me overcome challenges encountered during the research and implementation stages.

I would also like to thank all individuals who participated in the user feedback, prototype evaluation, data collection, and testing activities. Their opinions, observations, and suggestions were valuable in evaluating the practicality and usability of the proposed drain monitoring and blockage detection solution.

Finally, I would like to express my heartfelt gratitude to my family, friends, and everyone who supported and encouraged me throughout this research journey. Their continuous motivation, patience, and support were essential in helping me complete this research successfully.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 4

1.4 Research gap 5

1.5 Aim and objectives 6

1.6 Research questions or hypotheses 8

1.7 Scope limitations and assumptions 9

1.8 Individual contribution 10

1.9 Thesis structure 11

2 LITERATURE REVIEW AND RELATED WORK 13

2.1 Review method and scope 13

2.2 Domain and theoretical foundations 14

2.3 Related research 18

2.4 Existing systems products and techniques 21

2.5 Gap synthesis and conceptual framework 23

3 RESEARCH METHODOLOGY 26

3.1 Research design 26

3.2 Data participants or study materials 27

3.3 Experimental or study procedure 29

3.4 Measures and evaluation metrics 32

Water distance error 32

Blockage detection performance 32

Communication performance 32

3.5 Data analysis 34
3.6 Ethics privacy security and responsible AI 34
3.7 Reproducibility and research artefacts 35
4 PRODUCT AND SOLUTION DESIGN 37
4.1 Requirements 37
4.2 Component architecture within the complete solution 39
4.3 Design alternatives and decisions 40
4.4 Detailed design 41
4.5 Standards and professional practice 43
5 IMPLEMENTATION INTEGRATION AND TESTING 44
5.1 Development environment and technology stack 44
5.2 Component implementation 44
5.3 Integration with the other components 46
5.4 Verification and software testing 47
5.5 Deployment and operation 49
6 RESULTS AND VALIDATION 51
6.1 Research results 51
6.2 Product and technical results 52
6.3 User or stakeholder validation 53
6.4 Integration results 53
6.5 Summary of findings 53
7 DISCUSSION 55
7.1 Interpretation of findings 55
7.2 Comparison with literature baselines and existing solutions 55
7.3 Research contribution 56
7.4 Product contribution 56

7.5 Limitations and threats to validity 57
7.6 Implications 57
8 CONCLUSION AND FUTURE DIRECTIONS 59
8.1 Conclusion 59
8.2 Achievement of objectives 59
8.3 Individual research and product contributions 60
8.4 Future directions 61
REFERENCES 62
GLOSSARY 65
APPENDICES 66
Appendix A Research instruments or extended results 66
Appendix B Academic integrity and AI use disclosure 67

LIST OF FIGURES

Figure 1: Architecture of the Drain Management component within Flood Guard 360 39
Figure 2 : Microcontroller boards used for the monitoring and blockage units..... 44

LIST OF TABLES

Table 1: Hardware, platforms, and communication in the reviewed studies 21
Table 2: Comparison of existing systems and the proposed system 23

Table 3: Hardware 27
Table 4: Software 27
Table 5: Test conditions 31
Table 6: Evaluation Measures 33
Table 7: Main prototype settings to record 36
Table 8: Functional Requirement 37
Table 9 : Non-Functional Requirement 38
Table 10 : Main Design Choices 40
Table 11 : Main Firmware Pin Mapping 46
Table 12 : Verification Checklist 47
Table 13 : Technical results and evidence still required 52
Table 14 : Objective achievement based on the supplied evidence 59
Table 15 : Prototype settings and illustrative worked example 66

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
GPIO	General-Purpose Input/Output
HTTP	Hypertext Transfer Protocol
IOT	Internet Of Things
JSON	JavaScript Object Notation
MAE	Mean Absolute Error
MCU	Microcontroller Unit
S1 / S2 / S3	Upstream, middle, and downstream sensor points
Wi-Fi	Wireless local-area network technology

1 INTRODUCTION

1.1 Overall project context

The research project, titled “Flood Guard 360: 24/7, Every Hour,” aims to develop a technology based system to support flood monitoring, early risk detection, and post flood management. Floods can cause serious damage to people, buildings, roads, and essential services. Heavy rainfall, poor drainage systems, and hidden blockages can increase the risk of flooding. During and after floods, people may also face difficulties getting reliable information, receiving essential supplies, and identifying possible health problems. To address these challenges, the project combines several related components to improve flood preparedness, monitoring, and emergency response.

Flood Guard 360 consists of four main components: **Drain Management, Flood Mapping, Ration Distribution, and Symptom Detection**. The Drain Management component monitors water levels, rainfall, and possible blockages in drains to identify conditions that may lead to flooding. The Flood Mapping component displays flood information and affected or high risk areas on a map, helping users understand where flooding may occur. The Ration Distribution component helps manage and organize the distribution of essential supplies to people affected by floods. The Symptom Detection component helps identify reported symptoms that may indicate health problems among affected people, supporting health awareness and timely follow-up.

These four components work together to support different stages of flood management. They help monitor environmental conditions, identify possible flood risks, organize relief supplies, and support people after flood events. By combining sensor data, map based information, and relief and health related services, the system aims to provide useful information to residents, relief teams, and relevant authorities. This integrated approach can improve access to important information, support better decision making, and help coordinate activities during flood emergencies.

Within the overall Flood Guard 360 project, the Drain Management component focuses on collecting real time water level and rainfall data and identifying possible

blockages by comparing water levels at multiple locations. ESP32-based devices collect data from sensors and send the measurements to a backend server for processing and storage. This information can be used to monitor drainage conditions, identify unusual changes in water levels, and study conditions that may lead to flooding. The component also supports the wider flood mapping and emergency management functions of the project by providing useful drainage information.

Overall, Flood Guard 360 aims to provide a coordinated solution for flood preparedness, monitoring, and post-flood support through its four main components. By bringing together drain management, flood mapping, ration distribution, and symptom detection in one system, the project aims to improve the management of flood related information and support better responses to the needs of affected communities. Through the use of technology and timely information, the system seeks to contribute to safer communities and more effective flood management practices.

1.2 Individual component context

This research is about the Drain Management component of Flood Guard 360. This is the part of the system that watches the drains. It collects information about the water inside the drains and gives it to the other parts of the project. The Flood Mapping component, for example, uses this information to show flood areas on a map.

The component does three main jobs. The first job is to measure the water level in a drain in real time. An ESP32 device with an ultrasonic sensor does this. The sensor sends out a sound wave and measures how long it takes to come back from the water surface. From this time, the device works out how far the water is from the sensor, and then how high the water is in the drain. The second job is to detect rain. A rainfall sensor on the same device does this. Knowing whether it is raining helps us understand the water level readings, because a rising water level during rain is normal, but a rising level without rain may be a warning sign.

The third job is to find possible blockages. For this, a second device based on the ESP32-S3 is used. It has three ultrasonic sensors placed at three points along the

drain: upstream, middle, and downstream. The device works out the water level at each point and compares the levels of neighboring points. When the water flows freely, the levels at neighboring points are close to each other. If something blocks the drain, the water builds up behind it, and the difference between two neighboring levels becomes large. If this large difference stays for some time, the system treats it as a possible blockage in the section between those two points.

Both devices connect to Wi-Fi and send their readings to a backend server using HTTP. The server saves the readings in a database. The blockage readings go through a separate API. The Flood Guard 360 dashboard and map can then use this saved data. In this way, the raw sensor readings become useful information, and residents, relief teams, and authorities can see where a drain looks abnormal before the water overflows.

1.3 Problem statement

Floods happen often in Sri Lanka and they cause great damage. In November 2025, floods affected more than 830,000 people in 25 districts [13]. Officials and engineers in Sri Lanka have said many times that blocked drains and canals, often filled with garbage, are one of the main reasons for floods in cities [14], [16]. A drain that is partly or fully blocked cannot carry much water. So the water can come out onto the roads and into houses, even when the rain is not very heavy.

At the moment, people check drains mostly by hand. Many councils do not have enough workers to watch the drains all the time, so the checking is not regular [2]. Checking by hand is also not real time. People usually act only after a drain is blocked or has already overflowed, and not before [3]. A blockage that starts during a storm may stay unnoticed until the water is already coming out. This means the people who are responsible for the drain get no early warning.

Technology can help, but the existing IoT systems solve only a small part of the problem. Most of them measure the water level at one point only, and they send an alert when the level goes above a fixed value [1], [3], [4], [6]–[9]. A high water level at one point shows that the drain is filling up, but it does not show the reason. The reason may be heavy rain, or it may be a blockage. It also does not show where in the drain the blockage is. Without this information, workers may need to walk along a long drain to search for the problem, and this wastes time while the water keeps rising.

There is also a problem with the quality of the readings. Ultrasonic sensors give good results when the water is calm, but they make more errors when the water surface is moving. In one study, the average error grew from about 3% in still water to about 15% when the water was moving [11]. Drains are also dirty places. Mud, waste, and rust can damage the sensors and cause false alarms or missed warnings [3]. If a system gives an alert from only one reading, it may give many false alerts. People then stop trusting the system and start ignoring its warnings.

The problem of this research is therefore the following. How can low cost IoT devices be used to watch the drain water level and rainfall in real time? And how can

the system find a possible blockage and its rough place, with enough reliability to avoid false alarms, so that residents and authorities can act in time?

1.4 Research gap

The studies reviewed in Chapter 2 show that IoT water level monitoring is already common. Low-cost boards and ultrasonic sensors can measure the water level with small errors when the water is calm [5], [6], [8]. The readings can be sent to dashboards and mobile apps [3], [6]. Alerts can be sent by SMS, Telegram, or a buzzer [3], [7]–[9]. Some studies work directly on drains and sewers [1]–[4]. Even so, the reviewed studies leave the following gaps.

Gap 1: Most systems use one point and one fixed limit**.** Most of the systems judge the condition of a drain from one reading at one point [1], [3], [4], [6]–[9]. When the reading goes above a fixed limit, they send an alert. Because they have only one point, they cannot tell where a blockage is.

Gap 2: Water levels at nearby points are not compared**.** The system in [2] puts sensors in several manholes, but it reports each manhole separately. It says it wants to find the place of a blockage, but its tests only cover the individual readings. The platform in [11] surveys the bottom of a canal from time to time and does not watch the canal all the time. None of these studies compares the water levels at neighboring points in real time to find a blockage.

Gap 3: Rain is rarely used together with drain monitoring. Only two of the reviewed systems include rain sensing [5], [12]. One of them was made for irrigation canals and the other for forecasting river floods. Neither of them tries to find blockages.

Gap 4: A single reading is trusted too much. Ultrasonic readings have more error when the water moves [8], [11]. Dirt and rust in drains can also harm the sensors [3]. But none of the reviewed studies describes a method that waits for several abnormal readings in a row before it reports a blockage. Such a method is a simple way to reduce false alarms.

Gap 5: There is little drain-level work for Sri Lanka. The Sri Lankan studies that were reviewed are about mud in canals [11] and about river flood forecasting [12]. They are not about single city drains that are connected to a larger flood management system.

This research tries to fill these gaps. The proposed system measures water level and rainfall. It compares the water levels at three points along a drain to find a possible blockage and the section where it is. It waits for several abnormal readings in a row before it reports a problem. It also sends its results to the Flood Guard 360 dashboard and map.

1.5 Aim and objectives

Aim. The aim of this research is to design, build, and test a low cost IoT system that measures drain water level and rainfall in real time, and also finds a possible blockage and its rough place. The system should give residents and authorities an earlier warning of drain problems inside Flood Guard 360.

Objectives.

- To study existing drain and flood monitoring systems and find their weak points in blockage detection. This gives a clear base for the research. It shows what the existing systems measure, how they send alerts, and what they miss.
- To design the structure of the Drain Management component. The design includes the sensor devices, the way the data is sent, the backend server, and

the link to the Flood Guard 360 dashboard and map. A clear design lets each part be built and tested separately.

- To build an ESP32 device that measures water level with an ultrasonic sensor and detects rain. This device is the main monitoring unit. It gives continuous readings of the water level together with the rain condition.
- To build an ESP32-S3 device with three ultrasonic sensors that find a possible blockage and its rough place. The sensors are placed at the upstream, middle, and downstream points. The device compares the water levels of neighboring points. This is the main technical part of the research.
- To add a confirmation step. The system reads the sensors every 10 seconds. It reports a blockage only when the abnormal difference is found in 3 readings in a row, which is 30 seconds. This reduces false alerts caused by waves or other shortchanges in the water surface.
- To build the data layer of the system. The measurements are sent over Wi-Fi using HTTP, saved in a database, and made available to the dashboard and map. This makes sure the information reaches the people who need it.
- To test the system. The tests cover how accurate the sensors are, how well the system finds blockages and their place, how many false alerts it gives, and how long the data takes to reach the dashboard.
- To write down the limits of the prototype and suggest future work. This includes ideas such as predicting the water level from past sensor and weather data.

1.6 Research questions or hypotheses

Research questions

- RQ1. How can the water levels measured by ultrasonic sensors at the upstream, middle, and downstream points be compared to finding a possible blockage and its rough place?
- RQ2. How can 3 abnormal readings in a row, taken 10 seconds apart, be used to confirm blockage and reduce false alarms when the water surface moves for a short time?
- RQ3. How can rain detection and water level data be used together to explain abnormal readings?
- RQ4. How can the readings be sent, saved, and shown in real time inside Flood Guard 360?

Each question is answered by a part of the system. RQ1 is answered by the three-point comparison (Objective 4). RQ2 is answered by the confirmation step (Objective 5). RQ3 is answered by the rain sensing in the main unit (Objective 3). RQ4 is answered by the data layer (Objective 6).

Hypothesis

If the difference between neighboring water levels stays abnormal for several readings in a row, the system will find a simulated blockage and its section better than a one point limit on a single reading, and it will give fewer false alerts.

1.7 Scope limitations and assumptions

Scope. This research covers the Drain Management component of Flood Guard 360. It includes the water level and rain monitoring with the ESP32 device, and the blockage detection with the three-sensor ESP32-S3 device. It also includes the sending of data by Wi-Fi and HTTP, the saving of data in the backend, and the data link to the Flood Guard 360 dashboard and map. The work is a prototype. It was tested on d ma model drain.

What is not covered. The other three components of Flood Guard 360 are Flood Mapping, Ration Distribution, and Symptom Detection. Other team members have already completed them, and they are not part of this thesis. This thesis uses only the data link to the dashboard and map. Water level and flood prediction are also not covered. They are suggested only as future work. The research also does not cover measuring the speed of the water, finding the type of a blockage, or using the system outdoors for a long time.

Limitations. The ultrasonic readings are less reliable when the water surface is moving [8], [11]. The blockage is only rough. The system can tell the section between two sensors, but not the exact spot. The system needs three abnormal readings in a row, so it takes at least 30 seconds to report blockage. It cannot report one faster. The prototype also needs Wi-Fi at the place where it is installed.

Assumptions. Wi-Fi is available at the monitoring place, and the backend server can be reached. Each sensor is fixed at a height that was measured, and the three sensors are placed along the direction of the water flow. When the flow is normal, the difference between neighboring water levels is small and steady. The system takes one reading every 10 seconds. If it finds an abnormal difference in 3 readings in a row, which means 30 seconds, it identifies a possible blockage. The system assumes that a shortchange in the water surface, such as a wave, does not last for 3 readings in a row, while a real blockage does.

1.8 Individual contribution

The individual contribution of this research is the design, building, and testing of the Drain Management component. Based on the studies reviewed, the main contributions are as follows.

- **A three-point comparison method.** The system compares the water levels at the upstream, middle, and downstream points. In this way it finds a possible blockage and the section where it is, and it does not depend on one reading at one point.
- **A confirmation step.** The system reports a blockage only after the abnormal difference has stayed for several readings in a row. This reduces the false alarms that come from a moving water surface.
- **One unit for level and rain.** The main unit adds rain detection to the ultrasonic water level measurement. This helps to explain abnormal readings.
- **A data layer with its own blockage API.** The data layer saves the readings and gives the drain information to the Flood Guard 360 dashboard and map.
- **A test of the prototype.** The results are given in Chapter 4.

1.9 Thesis structure

This thesis has eight chapters.

Chapter 1, Introduction, gives the background of the whole Flood Guard 360 project and of my own component, the Drain Management component. It also gives the problem statement, the research gap, the aim and objectives, the research questions, the scope, limitations and assumptions, and my individual contribution.

Chapter 2, Literature Review and Related Work, looks at what other researchers and existing systems have done. Section 2.1 explains how the studies were found and chosen, and what the review covers. Section 2.2 explains the basic ideas behind this work, such as how drains carry water, how ultrasonic sensors measure distance, and how IoT devices send data. Section 2.3 reviews related research. Section 2.4 looks at existing systems, products, and techniques. Section 2.5 puts the gaps together and shows how this research fills them.

Chapter 3, Research Methodology, explains how the research was done. Section 3.1 gives research design. Section 3.2 describes the data, the materials, and the test setup used. Section 3.3 explains the steps of the experiments. Section 3.4 explains the measures used to judge the system, such as sensor error, detection of blockages, false alerts, and data delay. Section 3.5 explains how the results were analyzed. Section 3.6 explains ethics, privacy, and security points, and the use of AI if any. Section 3.7 explains how someone else can repeat the work, using the code, the circuits, and the settings.

Chapter 4, Product and Solution Design, explains the design of the system. Section 4.1 lists the requirements. Section 4.2 shows where my component fits inside the complete Flood Guard 360 system. Section 4.3 explains the other design choices that were considered and why the final ones were chosen. Section 4.4 gives the detailed design of the hardware, the blockage detection method, and the backend. Section 4.5 explains the standards and good practices that were followed.

Chapter 5, Implementation, Integration and Testing, explains how the design was built and checked. Section 5.1 describes the tools, boards, and software used. Section 5.2 explains how the component was built. Section 5.3 explains how it was joined to

the other components of Flood Guard 360. Section 5.4 explains the testing. Section 5.5 explains how the system is set up and run.

Chapter 6, Results and Validation, shows what was found. Section 6.1 gives the research results. Section 6.2 gives the product and technical results, such as sensor accuracy and blockage detection. Section 6.3 gives feedback from users or stakeholders. Section 6.4 gives the results of joining my component to the other components. Section 6.5 gives a summary of the findings.

Chapter 7, Discussion, explains what the results mean. Section 7.1 interprets the findings. Section 7.2 compares them with the studies and existing systems from Chapter 2. Section 7.3 explains the research contribution. Section 7.4 explains the product contribution. Section 7.5 explains the limitations and anything that may affect how far the results can be trusted. Section 7.6 explains what the results mean for residents, relief teams, and authorities.

Chapter 8, Conclusion and Future Directions, closes the thesis. Section 8.1 gives the conclusion. Section 8.2 shows whether each objective was achieved. Section 8.3 states my individual research and product contributions. Section 8.4 suggests future work, such as predicting water levels from past data.

The references and appendices come after Chapter 8.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

Purpose. This chapter looks at what other researchers and existing systems have done about drain monitoring and flood warning. It shows what is already known, and where the gaps are.

How the studies were found. The studies were collected from IEEE Xplore through the SLIIT library, from IOP Publishing conference series, from a university repository, and from an Indonesian journal.

What was included. A study was included if it met these points:

- It used sensors to measure water level, rainfall, or other drain conditions.
- It sent the data to a server, a phone, or an alert system.
- It was published between 2013 and 2026, and most are from 2018 or later.

The review has 12 technical studies. They fall into four groups:

- Drain and sewer systems [1]– [4]
- An irrigation canal system that also measures rain [5]
- River and dam flood warning systems that use ultrasonic sensors [6]– [9]
- Sri Lankan studies and a large flood warning framework [10]– [12]

Four more sources [13]– [16] give background about floods and blocked drains in Sri Lanka. They are mostly news reports and one conference paper.

Limits of this review. This is a focused review, not a full systematic review. It covers 12 technical studies. So, when this thesis says something is “not described in the reviewed studies”, it means only these 12 studies. It does not mean nobody in the world has done it.

2.2 Domain and theoretical foundations

This section explains the basic ideas behind the research.

Drains, blockages, and floods

A drain carries rainwater away from roads and houses. A flood happens when the amount of water is more than what a river or drain can carry [9]. A blocked drain carries less water. In Sri Lanka, officials and engineers say that blocked drains and canals, often full of garbage, are a main reason for city floods [14], [16]. Mud and floating waste also reduce the space in a canal and raise the flood risk [11]. Standing water in blocked drains is also a health problem, because it becomes a place where mosquitoes breed [4].

When something blocks a drain, the water cannot pass easily. It builds up behind the blockage. So, the water level on the upstream side of the blockage becomes higher than the water level after it. This is the main idea behind the three-point method in this thesis. It is a basic fact about flowing water. If your supervisor wants a source, a hydraulics textbook can be added here.

Measuring water level with an ultrasonic sensor

An ultrasonic sensor sends out a sound wave that people cannot hear. The wave hits the water surface and comes back. The sensor measures the time the wave takes. The distance is the speed of sound multiplied by the time, divided by two, because the wave goes to the water and back [5], [9]:

$$d = (t \times c) / 2 \quad (2.1)$$

Here d is the distance to the water, t is the round trip time, and c is the speed of sound. In air, [9] uses 340 m/s. The speed of sound changes a little with air temperature. If the sensor is fixed at a known height H above the bottom of the drain, the water level is

$$L = H - d \quad (2.2)$$

Ultrasonic sensors are popular for water level work for three reasons. They measure without touching the water, so mud and rust do not harm them as much as they harm

contact sensors. They were cheap. And they are easy to connect to a small board [5]–[9].

They also have limits.

- Every sensor has a minimum and a maximum range. In [5], the sensor reads well above 3 cm, but gives wrong values near 450 cm. In [9], the waterproof JSN-SR04T works between 20 cm and 600 cm. In [6], values above 233 cm were not reliable.
- The sensor works best when the water surface is flat and calm [8]. When the surface moves, the error grows. In [11], the average error was about 3% in still water and about 15% when the water was moving.
- Sensors in drains can be hurt by dirt, rain splashes, and rust. These can cause false alarms or missed warnings [3], [9].

These limits are the reason this thesis does not trust a single reading, and why it checks several readings in a row.

IoT devices and communication

IoT means small devices that connect to the internet and send data. The ESP32 is a low cost board with built in Wi-Fi. It is used in many of the reviewed systems [3], [4], [5], [11], [12]. A board like this reads the sensors, does simple calculations, and sends the results to a server.

The reviewed studies use different ways to send the data:

- **Wi-Fi to a cloud platform** such as Blynk, Thing Speak, or Firebase [1], [3], [4], [6], [12]
- **LoRa and MQTT** to a Things Board dashboard, for longer distances [5]
- **SMS through a GSM module,** which works where there is no internet but is slower [7]– [9]
- **XBee radio links** between units [8]

Wi-Fi with HTTP, used in this thesis, is simple. It works with a normal web backend. It needs Wi-Fi at the place of installation.

The studies also take readings at different speeds. In [6], the data is updated every 15 seconds. In [12], it is sent every 2 minutes to avoid heavy data traffic. This thesis uses one reading every 10 seconds.

Alert levels and thresholds

Most of the reviewed systems use thresholds. A threshold is a fixed number. When the water level passes it, the system changes its warning level or sends an alert. For example, [5] has four standby levels from Good to Warning. In [7], alert levels (Normal, Moderate, Critical, Emergency) decide how often SMS messages are sent. In [8] and [9], the water height is also divided into standby or alert levels. In [3], an alert starts when the water level is above 30 cm.

The threshold method is simple and works fast. Its weakness is that it uses one number from one point. It cannot tell the reason for a high level, and it cannot tell where the problem is. Unlike systems that use a fixed water level threshold at a single point, our approach establishes a threshold based on measurements taken at two monitoring points. The distance between the sensor's reference position near the riverbank and the water surface is measured to identify the normal water level and define a critical threshold. When the water rises and reaches the predefined critical level, the system identifies a potential flood condition and can trigger an alert. This approach helps monitor changes in water level at different points and provides an early indication of possible flooding.

Rainfall sensing

Rain adds to the water in a drain, so rain data helps to explain the water level. In [5], a tipping-bucket rain gauge was used. It measures rain in millimeters, and its errors were 0.97% and 1.62%. In [12], a raindrop sensor module was used to check the strength of the rain. Some systems also use weather data from official agencies [5].

Early warning systems

An early warning system collects data, finds signs of danger, and tells people in time to act. Large frameworks such as Urban Flood [10] use sensor networks, AI to find unusual readings, and computer simulations to help decision makers. The same basic idea, on a much smaller scale, is used in this research.

2.3 Related research

This section describes each reviewed study in turn.

Drain and sewer monitoring systems

Rajasekar et al. [1] made a drain block detection and controlling system. It uses an ultrasonic sensor, a water level sensor, and a GPS module on a NodeMCU. The two level sensors are compared with each other. The water level is also compared with normal, warning, and critical values. If the level is above the warning value, the system gives notice of a possible blockage or high water. If it is above the critical value, it tells the maintenance staff and uses the GPS position to show where the drain is. The data is saved in Google Firebase. The paper gives the design and the steps, but it gives no numbers for accuracy or response time. The blockage is judged from the water level at one node only.

Gawali et al. [2] made a drainage monitoring system where each manhole cover is a sensor node. Each node has an ultrasonic level sensor, a gas sensor, an air pressure sensor, and a sensor that shows if the manhole is open. The authors built a model drain 16 feet long with three manholes and made four sensor nodes. The server shows the readings and marks them in red when, for example, the water is above 50% of the drain height, a manhole is open, or the methane level is high. The introduction talks about finding the exact place of blockages. But the experiments only show the separate readings, and the paper gives no accuracy numbers.

Jayapriya et al. [3] made a drainage monitoring and alert system on an ESP32. It uses an ultrasonic sensor, a DHT11 sensor for temperature and humidity, and gas sensors. When a reading goes over its limit, a buzzer sounds, and the Blynk app shows a message. For water level, the limit is 30 cm. The system was tested in simulated drains. The authors say that the system does not work well where internet and power are poor. They also say that sensors in wet drains can lose accuracy because of dirt and rust, which can cause false alarms or missed warnings.

Chakraborty et al. [4] made a system for waterlogging control on an ESP32. It uses a water level sensor, an MQ2 gas sensor, a DHT11 sensor, and a BMP180 pressure sensor. When the water level passes a set value (about 60 mm in the prototype

results), two servo motors open the gates of the drain. The readings are shown on a Blynk dashboard. The paper shows tables of readings and the gate positions. It does not give accuracy or detection numbers.

Canal monitoring with rainfall

Hadi et al. [5] made a monitoring system for irrigation canals. An ESP32 reads a rain gauge, an ultrasonic sensor, and a flow meter. The data goes by LoRa to a gateway, and then by MQTT to a Things Board website. Telegram messages are sent when the condition reaches the alert level. The rain gauge errors were 0.97% and 1.62%. The ultrasonic sensor had an average error of 0.58%. The average delay to reach Things Board was about 1 second. This is the closest system to this thesis because it uses water level and rain together. But it was made for farm canals, and it does not look for blockages.

Ultrasonic flood warning systems

Diriyana et al. [6] used a NodeMCU ESP8266 and an HC-SR04 sensor. The data goes to Thing Speak every 15 seconds, and warnings go to a Telegram bot. The average error was 1 cm (0.78%). The data reached Thing Speak in about 1 second and Telegram in about 5 seconds.

Natividad and Mendez [7] put two solar powered ultrasonic stations on river basins in the Philippines. Stations send data by GSM to a server. The server sends SMS alerts, updates a website, and answers SMS questions from the public. Fuzzy logic is used for the alert status. The paper does not give accuracy numbers.

Indrasari et al. [8] used two HC-SR04 sensors and a flow sensor, with XBee and GSM links, for the Katulampa Dam in Indonesia. The working range was 14 cm to 250 cm, with the largest error of 4.33%. SMS messages arrived in 8.2 to 33.3 seconds. The authors found that using two sensors gave better readings when the water was moving.

Dswilan et al. [9] tested a waterproof JSN-SR04T sensor. It worked from 20 cm to 600 cm, with a very good straight-line fit ($R^2 = 0.9996$). Alerts go by SMS through a SIM900A module. The error was 0.49% with a buoy and 1.1% without one. But the

authors advise not to use a buoy in the field, because it is not stable when the water flow increases.

Sri Lankan studies and larger frameworks

Dissanayake et al. [11] made a floating platform to survey mud in urban canals in Sri Lanka. It has an underwater ultrasonic sensor, an ESP32, and a conveyor to collect floating waste. The readings are placed on a 6 × 5 grid to make 3D maps of the canal bottom. A risk class is given from the loss of canal capacity: low risk below 20%, moderate from 20% to 50%, and high above 50%. The authors say these limits are only rough guides. The sensor error was about 1.0% in a water container and 3.2% in a well when still, but about 15% when moving. The system checks a canal from time to time. It does not watch a drain all the time.

Ilukkumbure et al. [12] made an early warning system for the Kalu River basin. An ESP32 device has a DHT11 sensor, a raindrop sensor, and an AJ-SR04T ultrasonic module. It sends data to Firebase every 2 minutes. Machine learning models predict rain and water level. The water level model, an artificial neural network, got 84.3% accuracy on test data. The public can also send weather reports. The system works at river level. It does not cover drains or blockages.

Pengel et al. [10] describes Urban Flood, a large early warning framework. Sensors in dikes send data to a system that finds unusual readings with AI, and computer models estimate the chance of failure and how floods may spread. The results go to a decision support screen. It shows how real-time sensor data can support decisions. But it was made for dikes and levees, not for city drains.

2.4 Existing systems products and techniques

This section looks at the platforms and techniques that appear in the reviewed studies. It does not compare commercial products. The reviewed studies are research prototypes.

Platforms and communication used,

Table 1: Hardware, platforms, and communication in the reviewed studies

Study	Main board	Water level sensor	Communication	Platform or alert
[1] Rajasekar	NodeMCU	Ultrasonic + level sensor	Wi-Fi	Firebase, notices
[2] Gawali	Not named	Ultrasonic	PoE / Wi-Fi network	Server screen
[3] Jayapriya	ESP32	Ultrasonic	Wi-Fi	Blynk, buzzer
[4] Chakraborty	ESP32	Water level sensor	Wi-Fi	Blynk, servo gates
[5] Hadi	ESP32	Ultrasonic	LoRa, MQTT	ThingsBoard, Telegram
[6] Diriyana	NodeMCU ESP8266	HC-SR04	Wi-Fi	ThingSpeak, Telegram
[7] Natividad	Arduino	Ultrasonic	GSM	SMS, website
[8] Indrasari	Microcontroller	2 × HC-SR04	XBee, GSM	SMS, buzzer, LCD
[9] Dswilan	Arduino Uno	JSN-SR04T	GSM	SMS
[10] Pengel	Sensor network	Dike sensors	Internet / cloud	Decision screen
[11] Dissanayake	ESP32 + Arduino	Underwater ultrasonic	Wi-Fi	Blynk, Python GUIs
[12] Ilukkumbure	ESP32	AJ-SR04T	Wi-Fi	Firebase, app, SMS

Techniques used to decide that something is wrong

The studies use these techniques:

- **Fixed thresholds.** The system compares one reading to one or a few numbers. This is the most common method [1], [3]– [9]. It is simple and fast, but it cannot give the cause or the place.
- **Fuzzy logic.** The system turns alert levels into a warning status using simple rules [7]. This helps with decisions, but it still starts from a single-point level.
- **Machine learning.** Models such as the artificial neural network in [12] predict rain and water level from past data. They need a lot of past data and do not detect a blockage as it forms.
- **AI anomaly detection.** The large framework in [10] learns what normal readings look like and flags unusual ones. It needs a lot of data and computing power, and it was made for dikes.
- **Automatic response.** In [4], servo gates open when the water is high. This acts on the water level, but it does not find the cause.
- **Using two sensors for better accuracy.** In [8], two sensors on the same spot gave better readings on a moving surface. This improves reading at one point. It does not compare different points.

2.5 Gap synthesis and conceptual framework

Comparing the reviewed systems with the proposed system

Table 2: Comparison of existing systems and the proposed system

Study	Water level	Rain sensing	Compares nearby points	Blockage detection	Live dashboard or app	Alerts	Numbers reported
[1] Rajasekar	✓	✗	✗	Threshold warning	✓ (Firebase)	✓	✗
[2] Gawali	✓	✗	Separate nodes only	Proposed, not tested	✓ (server)	✓	✗
[3] Jayapriya	✓	✗	✗	✗	✓ (Blynk)	✓	✗
[4] Chakraborty	✓	✗	✗	✗	✓ (Blynk)	✗ (gate control)	Partly
[5] Hadi	✓	✓	✗	✗	✓(Things Board)	✓ (Telegram)	✓
[6] Diriyana	✓	✗	✗	✗	✓(Thing Speak)	✓ (Telegram)	✓
[7] Natividad	✓	✗	Two river stations	✗	✓ (website)	✓ (SMS)	✗

[8] Indrasari	✓	✗	✗	✗	LCD only	✓ (SMS, buzzer)	✓
[9] Dswilan	✓	✗	✗	✗	✗	✓ (SMS)	✓
[10] Pengel	✓ (dikes)	–	✓ (dikes)	✗	✓	✓	✗
[11] Dissanayake	✓ (canal bottom)	✗	Grid survey	Capacity loss risk only	✓ (Blynk, GUIs)	✗	✓
[12] Ilukkumbure	✓	✓	✗	✗	✓ (app, SMS)	✓	✓
Proposed system	✓	✓	✓ (3 points)	✓ (3 readings in a row)	✓ (Flood Guard 360)	✓	✓

Summary of gaps,

From the review, these gaps remain:

- **One point and one limit.** Most systems judge a drain from one reading at one point [1], [3], [4], [6]– [9]. They cannot tell the place of blockage.
- **No comparison between nearby points.** The system in [2] reports each manhole by itself. The system in [11] checks a canal from time to time. None compares nearby water levels in real time to find a blockage.

- **Rain is rarely used with drain monitoring.** Only [5] and [12] use rain sensing. One is for irrigation canals, and the other is for river forecasting. Neither finds blockages.
- **A single reading is trusted too much.** Error grows when the water moves [8], [11], and dirt can harm the sensors [3]. None of the studies wait for several abnormal readings in a row.
- **Little drain level work for Sri Lanka.** The Sri Lankan studies are about canal mud [11] and river forecasting [12], not single city drains linked to a bigger flood system.

Conceptual framework

The proposed system answers these gaps in three steps. Each step links to a research question from Chapter 1.

Step 1: Measure (RQ3, RQ4). An ESP32 unit measures water level and rain. An ESP32-S3 unit measures the water level at three points: upstream, middle, and downstream. The system takes one reading every 10 seconds.

Step 2: Decide (RQ1, RQ2). The system turns each distance into a water level. It compares the levels of neighboring points. If the difference is abnormal, it marks that reading as abnormal. If 3 readings in a row (30 seconds) are abnormal, the system reports a possible blockage in the section between those two points. The rain reading gives extra context.

Step 3: Share (RQ4). The readings and blockage status go over Wi-Fi to a backend with HTTP. They are saved in a database. Blockage data goes through its own API. The Flood Guard 360 dashboard and map show the results.

3 RESEARCH METHODOLOGY

3.1 Research design

This research is a design and builds study with experiments. It has three steps.

- **Build.** A prototype of the Drain Management component is built with ESP32 boards, ultrasonic sensors, and a rain sensor.
- **Test.** The prototype is tested in different water conditions, and the readings are recorded.
- **Check.** The results are compared with what the system should do, for example how close the readings are to real measurements, and whether the system finds a blockage or flood correctly.

The research uses real measurements from the prototype. It does not use a survey or a questionnaire. The system uses fixed rules and does not use machine learning, so there is no training data.

The prototype has two parts,

- **Station unit.** One ESP32 board has one ultrasonic sensor and one rain sensor. Together they work as a station. The station measures the water level and the rain for one specific place. The prototype uses the Putupaula station.
- **Blockage unit.** A second ESP32-S3 board has three ultrasonic sensors. They are used only for blockage detection.

The two research ideas that are tested are:

- A threshold rule that tells normal water from a possible blockage or a possible flood, and
- A confirmation rule: one reading every 10 seconds, and 3 abnormal readings in a row (30 seconds) before the system reports a problem.

3.2 Data participants or study materials

Participants

There are no human participants in the experiments. The tests only use the hardware and the water.

Table 3: Hardware

Item	Number	Use
ESP32-WROOM-32U board	1	Main station for water-level and rainfall monitoring.
ESP32-S3 board	1	Blockage unit with three ultrasonic sensors.
Ultrasonic sensor	4	1 for the station, 3 for blockage detection (model: HC-SR04)
Rain sensor	1	Detect rain at the station**.**
Breadboard and wires	2	Join the parts without soldering.
Power supply	As needed	USB power source or power bank for each unit during testing.
Hotspot	1	Gives the boards a link to the backend.

Table 4: Software

Part	Tool
Board program (firmware)	Arduino (Arduino IDE, C++)
Backend	MERN stack: MongoDB, Express, React, Node.js
Database	MongoDB
Python	Model training.

The MERN stack is a set of free tools. It uses JavaScript for the whole web part. Express and Node.js make the backend and its API. React makes web pages and graphs. MongoDB saves data.

Data collected,

Each time a unit takes a reading, it records:

- Distance from the sensor to the water surface (cm)
- Water level and gap left worked out from the distance (cm)
- Rain status from the rain sensor
- Station and sensor ID
- Time and date of the reading
- Status

The system takes one reading every 10 seconds.

Test setup,

A prototype was developed to test the drain monitoring and blockage detection system. The prototype consists of two main parts: a monitoring station and a blockage detection unit. The monitoring station includes one ultrasonic sensor to measure the water level and a rainfall sensor to detect rain. The blockage detection unit consists of three ultrasonic sensors placed at different positions to monitor water levels and identify possible blockages.

To test the monitoring station, a water cup was placed below the ultrasonic sensor. Water was added to and removed from the cup to simulate changes in water level. The sensor readings were used to monitor the water level and identify possible flood risks. The flood warning levels provided by the Irrigation Department for the relevant monitoring stations were used as a reference for identifying different flood risk levels. Rainfall detection was tested by spraying water near the rainfall sensor to simulate rain and observe its response.

To test the blockage detection unit, water cups were placed below the three ultrasonic sensors. Water was added to and removed from the cups to create different water levels at the three monitoring points. The system compared the measured water levels

to identify unusual differences between neighboring points that could indicate a possible blockage. These tests helped evaluate whether the prototype could monitor water-level changes and detect possible blockages under control conditions.

3.3 Experimental or study procedure

Measuring the water level

An ultrasonic sensor sends a sound wave to the water. It measures the time the wave takes to come back. The distance is [5], [9]:

$$d = (t \times c) / 2 \quad (3.1)$$

Here d is the distance, t is the round-trip time, and c is the speed of sound. Each sensor is fixed at a reference position above the drain. The system compares the measured distance with the distance for normal water.

Threshold rule

The threshold is found from measurements at the monitoring points. In the example used in this research, the sensor is 60 cm above water, and the gap between the water surface and the top of the riverbank is 20 cm. The threshold is set between 0 and this gap, because if the water rises the whole gap, it reaches the bank and a flood starts. When the water goes past the threshold but has not reached the bank, the situation is a possible blockage.

Confirmation rule

The system takes reading every 10 seconds. A reading is abnormal when it is past the threshold. The system reports a problem only when 3 abnormal readings come in a row, which is 30 seconds.

There is a reason for this. A single abnormal reading can come from something floating on the river, such as a leaf or a bottle. This kind of event is short. A real blockage or flood stays for much longer. So, 30 seconds of the same abnormal readings is strong evidence that the problem is real. A moving water surface also causes more sensor error [8], [11], and the 3 in a row rule reduces this problem.

The number of readings in a row (called N) can be changed in the code. A bigger N removes more false alerts. But it also makes the system slower. With a 10 second interval, N = 3 means at least 30 seconds, and N = 5 means at least 50 seconds. The best N is a balance between fewer false alerts and faster warnings.

Test steps

Each test follows these steps.

- **Build and connect** the hardware and connect each board to Wi-Fi and to the backend.
- **Set the reference positions** and the thresholds. Measure the real distance with a ruler or tape.
- **Run the test conditions** below.
- **Record** the readings, the statuses, and the time. The backend saves them in MongoDB, and the web page shows them as real time graphs.
- **Compare** the system's result with the real situation.

Table 5: **Test conditions**

Test	**What is done**	**What should happen**
T1. Normal flow	Water stays at its normal level	Status stays normal
T2. Rain	Rain, or water poured to act as rain	Rain is detected, and the level changes
T3. Blockage	A fake blockage is placed in the drain	Possible blockage is reported after 3 abnormal readings
T4. Flood	The water rises to the bank	Possible flood is reported
T5. Floating object	A leaf or bottle passes under a sensor	No blockage report.
T6. Accuracy	The water is set to known distances	Sensor reading is close to the real distance.

3.4 Measures and evaluation metrics

The system should be evaluated using measurements that can be checked against a ruler, a known test condition, or a backend log. The metrics below can be used during prototype tests. The final values must come from the recorded tests, not from estimates.

Water distance error

For each test, compare the sensor distance with the distance measured by a ruler or measuring tape. The absolute error is the difference between the two values.

Absolute error (cm) = |sensor distance − reference distance|

The mean absolute error (MAE) is the average of the absolute errors across all valid samples. Percentage error can also be calculated when the reference distance is not zero: percentage error = (absolute error ÷ reference distance) × 100.

Blockage detection performance

During a controlled test, record whether a blockage was introduced and whether the system reported it. A true positive is a blockage test correctly reported as a blockage. A false positive is a blockage alert when no blockage was introduced. A false negative is a blockage that was introduced but not detected. A true negative is a normal-flow test with no blockage alert.

Detection rate (%) = true positives ÷ (true positives + false negatives) × 100. False-alarm rate (%) = false positives ÷ (false positives + true negatives) × 100. These values should only be calculated after the number of tests, and their outcomes have been recorded.

Communication performance

Record the total number of HTTP requests, the number of successful responses, and any failed requests. API success rate (%) = successful requests ÷ total requests × 100. If device and server timestamps are recorded, transmission delay can be calculated from them. Otherwise, measure and report the HTTP response time available from the device log, and describe it as response time rather than one-way network delay.

Table 6: Evaluation Measures

Measure	How to measure it	Evidence to record
Distance MAE	Average absolute difference between sensor and ruler measurements	Reference distance, sensor distance, sample count
Percentage error	Absolute error divided by non zero reference distance, multiplied by 100	Error for each distance test
Blockage detection rate	Correctly detected introduced blockages divided by all introduced blockages	True positives and false negatives
False-alarm rate	False alerts divided by all normal tests	False positives and true negatives
API success rate	Successful HTTP posts divided by attempted posts	HTTP codes and total attempts
Response time	Time from sending a request until the HTTP response is received	Serial log timestamps or measured elapsed time

A small prototype test can show whether the design works under controlled conditions. It cannot, by itself, prove that the same error rates or detection rates will apply to every real drain or river.

3.5 Data analysis

The analysis uses the readings collected during the water level, rainfall, and blockage tests. First, the reference measurements and sensor readings are placed in a table and checked for missing values or readings outside the sensor range. Readings that have no echo are recorded as sensor errors and are not treated as valid distances.

For water level tests, calculate the absolute error for each known distance and then calculate the mean absolute error and percentage error where suitable. Plotting the reference distance against the sensor distance can help show whether the sensor follows the expected trend. The test should also note whether readings change when the water surface moves.

For blockage tests, label each run as normal flow or simulated blockage, and record the section where the blockage was introduced. Compare this known condition with the reported result. Count correct detections, missed detections, and false alerts. The result should state the number of trials as well as the percentage so that readers can judge how much evidence is available.

For communication tests, review the serial log and backend records together. The request is not counted as successfully integrated only because the device printed a JSON message. The HTTP result should be successful, and the matching record should be visible in the database. The main sensor route and blockage route should be checked separately.

Because the supplied materials do not contain a complete set of measured readings, this draft does not report invented accuracy or detection percentages. The final analysis must be updated with the actual test log before submission.

3.6 Ethics privacy security and responsible AI

The prototype tests described in this thesis use electronic sensors, water, containers, and software. They do not require personal details from members of the public. If real locations or user accounts are included in the dashboard, only the information needed for flood management should be collected, and access should be limited to authorized users.

The microcontrollers send data to the backend through HTTP on the test network. The current code includes network credentials and a device key directly in the firmware. These values must not be published in a thesis, a public code repository, or screenshots. Before real deployment, credentials should be changed and stored securely, and HTTPS should be used where supported. Device keys should be revocable and should not be shared between unrelated devices unless the backend design explicitly requires it.

A false alert can create unnecessary work, while a missed alert can delay a response. Therefore, the system should report its output as a possible blockage or a possible high-water condition, not as a guarantee that a flood will or will not occur. The prototype is intended to support human decisions and must not replace official warnings or decisions made by disaster management authorities.

The current sensor and blockage firmware uses fixed rules. It does not use a machine learning model or an AI symptom detection model. If machine learning is added later, the training data, validation method, errors, and limitations must be reported. The team must also follow the SLIIT rules for the disclosure of generative AI and other automated tools used in preparing the report.

3.7 Reproducibility and research artefacts

To allow another person to repeat the experiment, the following items should be kept with the project records: firmware source code, wiring diagrams, sensor model numbers, the measured mounting height of each sensor, the threshold values, test conditions, serial logs, backend responses, and the final test results. The firmware should be versioned so that the settings used during the reported tests can be identified.

The test environment should record the date, the test number, the sensor heights, the reference distances, whether rain was simulated, whether a blockage was inserted, and the result produced by the system. The same reference distances and test steps should be used when comparing different versions of the firmware.

The credentials in the supplied source files have been left out of this thesis. A public appendix should include only safe configuration examples and should never contain a Wi-Fi password, live device key, or other secret. A reader can repeat the work by adding their own local credentials and API address to a private configuration file.

Table 7: Main prototype settings to record

Setting	**Value in supplied firmware**	**What to verify before final testing**
Main unit sampling / HTTP interval	20 seconds	Confirm timing from the final firmware and serial log
Blockage unit sampling interval	10 seconds	Confirm three readings are collected at the intended interval
Blockage water level difference threshold	3.0 cm	Calibrate under normal flow and simulated blockage
Required consecutive abnormal readings	3	Confirm the alert is issued after three consecutive abnormal comparisons
Main unit reference height	700 cm in supplied code	Check against the real measured installation height; change if incorrect
Blockage sensor reference heights	21.00 cm, 21.49 cm, 19.98 cm	Measure and update for the actual sensor mounting
Rain analogue calibration	Dry 4095; wet 2595 in supplied code	Recalibrate for the actual rain module and ADC readings

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements describe what the Drain Management component should do, and the quality expected from it. The current work is a low-cost prototype. Some requirements, especially flood alerts based on the distance to the riverbank and full dashboard integration, require additional confirmation before they can be treated as complete.

Table 8: Functional Requirement

ID	Requirement	Priority / status
FR1	Measure the distance from the ultrasonic sensor to the water surface.	High – implemented in firmware
FR2	Convert the distance into a water level value using a measured sensor reference height.	High – implemented; reference height needs calibration
FR3	Read the rainfall module and estimate relative wetness.	High – implemented in firmware; does not measure rainfall in millimeters
FR4	Measure three water level points: upstream, middle, and downstream.	High – implemented in blockage firmware
FR5	Compare the water levels of neighboring points.	High – implemented in blockage firmware
FR6	Confirm a possible blockage after three consecutive abnormal comparisons.	High – implemented with a 3 cm difference setting
FR7	Send main-unit readings to the sensor-readings API.	High – firmware configured; backend success must be verified
FR8	Send blockage readings to a separate blockage API.	High – firmware configured; backend success must be verified

FR9	Show the readings and blockage status on the Flood Guard 360 dashboard.	High – integration requirement; confirm with end to end test
FR10	Give an early flood warning based on the measured gap to the riverbank.	Future extension unless this rule is added and tested in firmware

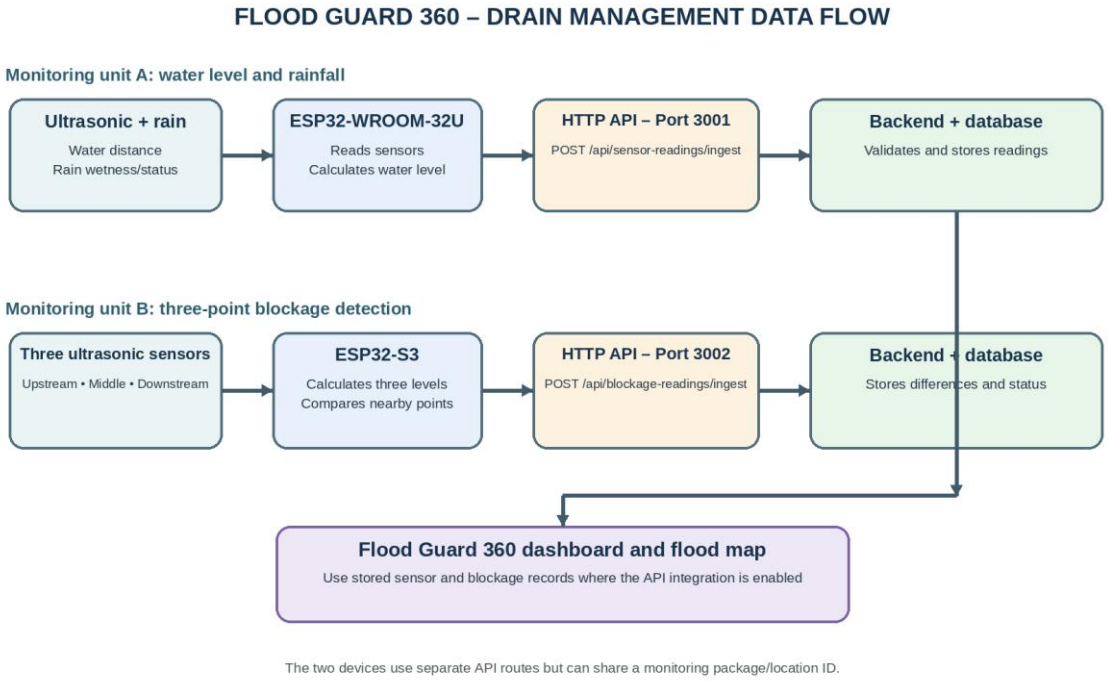
Table 9 : Non-Functional Requirement

ID	**Requirement**	**Reason**
NFR1	Use low cost and widely available parts.	Makes the prototype easier to build and repair.
NFR2	Use a clear data format and separate API routes.	Keeps the two monitoring units easier to manage.
NFR3	Report sensor errors instead of treating them as valid readings.	Avoids calculating a water level from a missing echo.
NFR4	Protect Wi-Fi credentials and device keys.	Prevents unauthorized access to the backend.
NFR5	Make the device easy to calibrate and maintain.	Sensor mounting heights and thresholds can differ by site.
NFR6	Use understandable status values.	Users need to know whether reading is normal, abnormal, or unavailable.
NFR7	Avoid making safety guarantees from the prototype.	The output is an aid and must be checked against real conditions.

4.2 Component architecture within the complete solution

The Drain Management component has two monitoring units. The main unit measures water level and rainfall at one location. The second unit compares water levels at three nearby locations to identify possible blockages. Each unit sends its records through a separate backend API route. Both sets of records can use the same package or location identifier because they relate to the same monitored place, while the separate routes keep the data types distinct.

Figure 1: Architecture of the Drain Management component within Flood Guard 360



The diagram shows the intended data flow. In an end-to-end test, each device must receive a successful HTTP response, each record must appear in the database, and the dashboard or flood map must display the matching values. The architecture diagram alone does not prove that every integration step is working.

4.3 Design alternatives and decisions

Several choices were considered during the design. The final choices were based on cost, ease of development, and the need to compare water levels at more than one point.

Table 10 : Main Design Choices

Design choice	Alternative	Decision and reason
Microcontroller	Arduino Uno with an external network module	ESP32 boards were selected because Wi-Fi is built in, so the devices can communicate with the web backend without a separate Wi-Fi module.
Water-level sensor	Contact float sensor or pressure sensor	Ultrasonic sensors were selected because they measure without touching the water and are easy to use in a small prototype. They still require protection and calibration.
Number of blockage points	One or two monitoring points	Three points were selected so the water level can be compared in two neighboring sections: S1–S2 and S2–S3.
Blockage decision	One measurement or fixed water-level value	The prototype checks level differences between neighboring points and requires three consecutive abnormal readings. This is intended to reduce alerts from short changes.
Communication	MQTT or GSM/SMS	HTTP was selected because it can post JSON records directly to the existing backend API. It depends on a working Wi-Fi network.
Prediction method	Machine learning model	A fixed rule method was used for the current blockage firmware because it

		can be tested without a labelled training dataset. Prediction from historical and weather data remains future work.

4.4 Detailed design

Water-level calculation

An ultrasonic sensor measures the distance from its mounting point to the water surface. The firmware estimates distance from the echo time. If t is the round trip travel time and c is the speed of sound, distance d is calculated as:

$$d = (t \times c) / 2$$

The division by two is needed because the sound wave travels to the water and then returns to the sensor. If H is the measured height of the sensor reference point above the bottom of the test container or drain, the estimated water depth L is:

$$L = H - d$$

The firmware limits negative levels to zero and does not allow a level higher than the configured reference height. For correct readings, H must be measured at the installation site. An incorrect H value causes all calculated levels to be wrong.

Rainfall sensing

The rain module provides a digital status and an analogue reading. The digital output indicates whether the module detects wetness, while the analogue output changes with the amount of water on the sensing surface. The supplied code converts the analogue value to a relative wetness percentage using dry and wet calibration values. This percentage is not the same as rainfall in millimeters and should not be described as a calibrated rain gauge.

Multi-point blockage detection

The ESP32-S3 unit reads three ultrasonic sensors. The sensors are referred to as S1 (upstream), S2 (middle), and S3 (downstream). For each point, the firmware

calculates a water level from the measured distance and the reference height stored for that sensor. It then calculates two absolute differences:

$$\text{Difference12} = |\text{Level1} - \text{Level2}|$$

$$\text{Difference23} = |\text{Level2} - \text{Level3}|$$

In the supplied code, a difference greater than 3.0 cm is considered abnormal. The system keeps a separate counter for each pair. If a pair has an abnormal difference for three consecutive readings, that section is marked as a possible blockage. The recorded location is between S1 and S2, between S2 and S3, or both sections if both conditions are confirmed.

This comparison provides an approximate section, not the exact position or type of blockage. A difference can also occur because of changes in channel shape, water flow, sensor height, or measurement error. The threshold must therefore be tested under normal flow and simulated blockages at the selected site.

Riverbank-gap flood threshold

For flood risk monitoring at a river or open channel, a separate rule can use the gap between the water surface and the top of the bank. The gap should be measured at the site, and a warning value should be set before the gap reaches zero. This rule is different from the 3.0 cm neighboring level difference used for blockage detection. The bank gap warning is a design extension and is not enforced by the supplied firmware, so it requires implementation and testing before it can be reported as a working flood alert.

Backend message design

The main unit posts a JSON message to the sensor readings route on port 3001. It includes the package identifier, water level in meters, the unit field, and the relative wetness value under the rainfall field. The blockage unit posts to the blockage readings route on port 3002. It includes the package identifier, each sensor distance, each calculated water level, both neighboring differences, the blockage Boolean, the approximate section, and the unit.

The two API routes should validate the package identifier and numeric values, reject invalid requests, and save the date and time of each accepted record on the server. The device key must be checked by the backend. No Wi-Fi passwords, live device keys, or private network addresses are included in this thesis.

4.5 Standards and professional practice

The prototype follows basic engineering practices: measurements use stated units, each sensor has a known role, configuration values are documented, and the test steps are designed to be repeatable. Water-level values and neighboring differences should always be labelled with the same units. The firmware converts water levels and differences from centimeters to meters in the JSON messages, while raw sensor distances are sent in centimeters. The backend schema must reflect this difference to avoid unit errors.

The device should be mounted securely and protected from rain, splashes, dirt, and accidental contact. Wires and the power supply should be kept away from standing water. Before field use, the complete enclosure, power supply, sensor range, and electrical safety should be reviewed. The current prototype has not been certified for outdoor or flood-service use, and no claim of compliance with a specific safety standard is made in this thesis.

Good software practice also requires input validation, safe handling of credentials, readable logs, and error handling when a sensor or network connection fails. The research report uses numbered references, labels tables and figures, and distinguishes implemented features from proposed extensions. These steps help readers understand the limits of the work and reproduce the tests.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The sensor firmware was developed in the Arduino IDE using C++. The firmware uses the ESP32 Wi-Fi library to connect to the local network and the HTTP Client library to send readings to a backend API. The current project backend is based on the MERN stack: MongoDB, Express, React, and Node.js. MongoDB is used for stored records, Express and Node.js provide API routes, and React presents the web interface.

The two monitoring units use different API routes. The main station is configured for the sensor-readings route on port 3001, while the blockage unit is configured for the blockage-readings route on port 3002. These are development settings and may differ from the production server. Private IP addresses and authentication keys are intentionally omitted from this report.

5.2 Component implementation

The implementation has two microcontroller units. The first measures water level and detects rain. The second compares water levels at three points. The development photos below show the boards used in the prototype; they do not show a finished weather proof field installation.



Figure 2 : Microcontroller boards used for the monitoring and blockage units

Main water-level and rainfall unit

The main unit uses an ESP32-WROOM-32U board, an ultrasonic sensor, and a rain sensor module. In the supplied firmware, the ultrasonic trigger pin is GPIO 27 and the echo pin is GPIO 14. The rain module digital output is read through GPIO 4 and the analogue output through GPIO 34. The onboard indicator LED uses GPIO 2. The firmware sends readings at a 20-second interval.

The ultrasonic reading is converted into a distance using the echo pulse time. The configured reference height is then used to estimate the water level. The rain module is read through its digital and analogue outputs. The code calculates a relative wetness percentage from the current analogue reading and the stored dry and wet calibration values. Before final testing, the reference height and rain calibration must be checked against real measurements.

Three-point blockage unit

The blockage unit uses one ESP32-S3 board and three HC-SR04 ultrasonic sensors. Sensor 1 measures the upstream point, Sensor 2 the middle point, and Sensor 3 the downstream point. In the supplied code, the trigger and echo pins are S1: GPIO 5 and GPIO 6; S2: GPIO 7 and GPIO 15; and S3: GPIO 16 and GPIO 17. The indicator LED is connected to GPIO 8.

The reference heights currently stored in the code are 21.00 cm, 21.49 cm, and 19.98 cm for S1, S2, and S3. These are prototype settings and must match the actual measured sensor mounting heights. The firmware takes readings every 10 seconds, calculates the water level at each point, finds the two neighboring differences, and confirms a possible blockage after three consecutive readings above the 3.0 cm threshold.

Table 11 : Main Firmware Pin Mapping

Unit	Part	ESP32 pin / setting	Function
Main station	Ultrasonic TRIG	GPIO 27	Starts the ultrasonic measurement
Main station	Ultrasonic ECHO	GPIO 14	Receives the returned pulse
Main station	Rain digital output	GPIO 4	Reads rain-detection status
Main station	Rain analogue output	GPIO 34	Reads the wetness signal
Main station	Indicator LED	GPIO 2	Shows transmission or status through flashes
Blockage unit	Sensor 1 TRIG / ECHO	GPIO 5 / GPIO 6	Measures upstream distance
Blockage unit	Sensor 2 TRIG / ECHO	GPIO 7 / GPIO 15	Measures middle-point distance
Blockage unit	Sensor 3 TRIG / ECHO	GPIO 16 / GPIO 17	Measures downstream distance
Blockage unit	Indicator LED	GPIO 8	Shows status and briefly flashes for a detected blockage

5.3 Integration with the other components

The main station and blockage unit use the same package identifier so that the backend can associate both data streams with one monitored location and package. They still use separate API routes because their data structures are different. The main unit sends the water level and relative rain-wetness value. The blockage unit sends the three raw distances, three calculated levels, neighboring differences, blockage status, and estimated section.

For integration to be considered complete, the backend must accept both routes, verify the device key, store each record, and return a successful HTTP status. The database must then contain the new record, and the dashboard/map should display the correct package and time. The tests should also confirm that one route does not accidentally store fields from the other data type.

The Flood Mapping component is intended to use the location information associated with the package. The current sensor messages shown in the supplied firmware do not include latitude or longitude directly, so the backend must link the package identifier to a location record before the readings can be plotted on a map. Any integration status should be reported from an actual end to end test.

5.4 Verification and software testing

Testing should cover sensor readings, the threshold logic, HTTP communication, and the database record. The test conditions in Chapter 3 can be repeated with known water distances and simulated blockages. Tests should be carried out one at a time so that a change in the result can be linked to a known condition.

Table 12 : Verification Checklist

Test ID	Verification action	Pass condition	Result to enter
VT1	Place water at a known distance from the main ultrasonic sensor.	Reported distance is close to the ruler measurement.	Code calculates distance and water level; measured error is not supplied.
VT2	Wet the rain module and compare digital/analogue readings.	The module changes state or wetness value as expected after calibration.	Code reads digital and analogue rain outputs and calculates wetness;

			dry/wet readings are not supplied.
VT3	Keep all three blockage points at similar water levels.	No blockage is confirmed during stable normal-flow tests.	Adjacent-level comparison and the three-reading rule are coded; the false-alert count is not supplied.
VT4	Create a water level difference greater than 3 cm between S1 and S2.	The system reports a possible blockage in S1–S2 only after three consecutive abnormal readings.	Configured rule: difference greater than 3 cm for three consecutive 10-second readings (about 30 seconds). Physical test result is not supplied.
VT5	Create a difference greater than 3 cm between S2 and S3.	The system reports a possible blockage in S2–S3 only after three consecutive abnormal readings.	Configured rule: difference greater than 3 cm for three consecutive 10-second readings (about 30 seconds). Physical test result is not supplied.
VT6	Disconnect or obstruct one sensor so that no echo is received.	The system reports a sensor error and does not use that sample as a valid comparison.	No echo triggers SENSOR ERROR and the sample is skipped; a fault-test result is not supplied.
VT7	Send a main-unit	A successful HTTP response	Main API POST is

	record to the sensor API.	is received and the record is stored.	coded; successful HTTP status and database record are not evidenced.
VT8	Send a blockage record to the blockage API.	A successful HTTP response is received and the record is stored.	Blockage API POST is coded; successful HTTP status and database record are not evidenced.
VT9	Open the Flood Guard 360 dashboard/map.	The correct location/package and current stored readings are displayed.	Dashboard/map integration is described; end-to-end verification evidence is not supplied.

5.5 Deployment and operation

For a controlled demonstration, secure the sensors above the test containers, measure each sensor reference height, connect the microcontrollers to a stable USB power supply, and connect them to a Wi-Fi network that can reach the backend. Start the backend before powering the devices. Check the serial output to confirm that each sensor produces a reading and that the device connects to Wi-Fi.

Before any outdoor installation, place the electronics in a suitable weather resistant enclosure, keep power connections away from water, secure the sensor brackets, and check that the ultrasonic path is clear. Confirm the expected sensor range and angle. Record the measured mounting height and recalibrate the rain module. The supplied

reference height of 700 cm in the main unit code must be confirmed against the real setup; it should be changed if it does not match the measured installation.

During operation, check the device connection, sensor values, HTTP response codes, and database timestamps. Clean the sensor faces where needed and repeat a known distance check after moving a sensor. If the network fails, remote monitoring may stop even if the device remains powered. A real deployment would need a clear process for connection recovery, maintenance, and checking any warning before acting on it.

6 RESULTS AND VALIDATION

6.1 Research results

The materials supplied for this thesis include microcontroller firmware and a description of controlled water level and blockage tests. They do not include a complete test log containing reference distances, sensor readings, alert counts, HTTP response times, or database screenshots. Therefore, this draft can report the functions present in the code, but it cannot give a reliable numerical accuracy or blockage detection result.

The code shows that the main station reads an ultrasonic distance and a rain module, calculates a water-level value and relative wetness, and attempts to send JSON data to the sensor-readings API every 20 seconds. The blockage unit reads three ultrasonic sensors every 10 seconds, calculates three water levels, compares S1–S2 and S2–S3, and uses a 3.0 cm difference threshold with three consecutive abnormal readings. These are implementation settings; their real-world accuracy and reliability must be established with measured tests.

The bank gap flood warning described in the design is not present in the supplied station firmware. It should not be reported as a tested flood alert unless the rule is implemented, connected to an alert output, and evaluated.

6.2 Product and technical results

The available source code provides evidence of the programmed functions listed in Table 13.

Table 13 : Technical results and evidence still required

Item	Confirmed from supplied firmware	Evidence needed for a final result
Main water level reading	Distance is converted to a level using a configurable reference height.	Reference distance versus sensor distance across repeated samples; confirm the 700 cm setting.
Rain sensing	Digital and analogue signals are read; analogue value is converted to relative wetness.	Dry and wet raw values, repeatability, and correct calibration for the installed module.
Three point comparison	Water levels are calculated for S1, S2, and S3; two neighboring differences are computed.	Readings from normal flow and each simulated blockage condition.
Blockage decision	A difference above 3.0 cm for three consecutive readings marks a possible blockage section.	True detections, missed detections, false alerts, and observed confirmation time.
HTTP communication	Firmware creates POST requests to separate API routes.	HTTP status codes, request totals, server response, and stored database records.
Flood map integration	Architecture supports linking readings to a monitoring package/location.	Evidence that the dashboard/map displays the correct stored records.
Bank gap flood warning	Described as a possible design extension but not	Implement and test the rules before describing it as complete.

	implemented in the supplied firmware.	

6.3 User or stakeholder validation

No formal questionnaire or human-participant study is described in the supplied methodology. For that reason, this thesis does not claim that residents, relief teams, or local authorities have validated the component. The prototype should first be tested technically. A later evaluation may ask intended users whether the readings, status labels, and location information are easy to understand and whether the information would help them decide when to inspect a drain.

If user feedback is collected later, describe who participated, how they were invited, what questions were asked, how consent was obtained, and how the answers were summarized. Do not include names or private information without permission.

6.4 Integration results

The firmware contains separate HTTP routes for the two data streams: the main sensor reading route and the blockage readings route. The messages share a package identifier, which can help the overall system connect each reading with a location. However, the supplied materials do not include the corresponding backend logs or a screenshot proving that both types of data were accepted, saved, and displayed.

6.5 Summary of findings

The supplied firmware implements the core sensor reading and rule based blockage comparison logic. It also defines separate HTTP routes for sending the two data types. The three point approach provides more information about the section where

water levels differ than a single sensor reading, but it can only indicate a possible blockage and cannot prove the cause of the difference. Numerical performance, false-alert reduction, backend success, and dashboard integration remain to be confirmed using recorded test evidence.

7 DISCUSSION

7.1 Interpretation of findings

The key design choice in this research is to compare water levels at neighboring points rather than depend only on one water level value. If the water builds up before an obstruction, the upstream level may be higher than the level after it. Comparing S1–S2 and S2–S3 gives a rough indication of which section needs attention. Requiring three consecutive abnormal readings helps avoid reacting to one shortchange in the water surface.

This method must be interpreted carefully. Water levels can differ for reasons other than blockage, such as a change in channel shape, local flow, sensor mounting height, or a faulty sensor. The absolute difference rule also does not account for the expected slope of a real channel. Calibration and tests under normal flow are therefore necessary before the threshold is used in an actual drain.

The rain sensor provides extra context, because rainfall can explain why the water level is rising. However, the present module estimates relative wetness rather than rainfall depth in millimeters. It cannot, by itself, distinguish all causes of a high-water level. The riverbank gap method is a separate idea for early flood warning and needs to be implemented and tested in firmware before conclusions can be drawn about its performance.

7.2 Comparison with literature baselines and existing solutions

The reviewed systems commonly use ultrasonic or water level sensors, fixed thresholds, a web dashboard, and an alert channel [1], [3]– [9]. These systems show that low-cost sensor readings can be sent to a remote interface. Some systems also include rainfall sensing [5], [12], while other studies focus on canal inspection or wider flood prediction [10]– [12].

The proposed component differs mainly by comparing water levels at three nearby points and requiring repeated abnormal readings before reporting a possible blockage. This design responds to the gap identified in Chapter 2: one reading at one

point can show that water is high but usually does not show which neighboring section may be affected. The difference is a design contribution, not yet a proven performance improvement. A fair comparison requires the same test conditions and measured values for detection rate, false alarm rate, and response time.

Unlike systems that use tipping bucket gauges or calibrated rain gauges [5], the present module uses a low cost rain sensor that reports relative wetness. It is therefore suitable for detecting whether the sensor is wet, but it should not be compared directly with rainfall measurements in millimeters without additional calibration.

7.3 Research contribution

The research contribution is a low cost design for drain monitoring that combines water level and rainfall sensing with a separate three-point blockage checking unit. It describes how to calculate water levels from ultrasonic distance measurements, compare neighboring points, and use repeated abnormal readings to reduce the effect of short changes.

The work also defines separate data routes for the main water level/rain sensor and the blockage unit. This separates different message formats while allowing both streams to be linked to a shared monitoring package. The final strength of these contributions should be supported with measured sensor errors and blockage tests.

7.4 Product contribution

The product contribution is a prototype that uses ESP32-based devices and existing web technologies. The main unit can measure water level and read a rain sensor; the secondary unit can compare water levels at three points. The firmware sends structured JSON messages to separate backend routes, allowing the data to be saved and later displayed by Flood Guard 360.

This component is intended to support the overall platform by providing useful information for drain inspection and flood-risk awareness. It does not independently manage ration distribution, detect health symptoms, or calculate a complete flood map. Those functions belong to other components of the wider project.

7.5 Limitations and threats to validity

The prototype has several limits. First, ultrasonic accuracy depends on sensor range, angle, the condition of the water surface, and the correct reference height. Second, only three points are compared, so the reported section is approximate, and the exact blockage position is unknown. Third, the 3.0 cm threshold may not suit every drain and must be calibrated for the geometry and normal flow of each site.

Fourth, the main unit uses a relative wetness percentage, not a calibrated rainfall rate. Fifth, the supplied firmware uses Wi-Fi and HTTP, so remote monitoring depends on network availability and a reachable backend. Sixth, the supplied source code has not been accompanied by complete numerical test logs in the materials reviewed for this draft. Claims about accuracy, alert reliability, API delivery, or the number of false alerts cannot be made until those records are analyzed.

The main unit also needs its reference height checked. The supplied code sets this value to 700 cm; if that does not match the actual sensor to reference distance, the water level estimate will be wrong. Finally, the prototype is not evidence of long term reliability in dirty, fast flowing, or outdoor drain conditions. It should not be used as the only source for life safety decisions.

7.6 Implications

For residents and local authorities, a system that reports changing water levels and possible blockage sections may help direct inspections to the places that need attention. The rainfall reading can provide context for a rising water level. If the data are linked to the Flood Guard 360 map, the project could help users see where monitoring is taking place and review recent conditions.

These benefits depend on reliable sensors, a working backend, clear status messages, and a suitable response process. The prototype can support inspection decisions, but workers must confirm the actual condition before taking action. Further field tests and feedback from intended users are needed before the component can be recommended for operational flood management.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This research developed the design and firmware for the Drain Management component of Flood Guard 360: 24/7, Every Hour. The component combines an ESP32-based main unit for water-level and rain sensing with a separate ESP32-S3 unit that compares three water level points to identify a possible blockage. The two units are configured to send their data to different backend API routes.

The three point comparison is intended to provide more information than a single point threshold because it can identify the section between neighboring sensors where water levels differ. The system requires three consecutive abnormal readings before it confirms a possible blockage. However, this method provides an approximate indication, and the threshold must be checked under normal and blocked conditions.

8.2 Achievement of objectives

Table 8.1 summarizes the objectives against the evidence currently available in this draft.

Table 14 : Objective achievement based on the supplied evidence

Objective	Current status	Evidence or remaining work
Review existing drain and flood-monitoring systems	Documented in Chapter 2	Check each reference and confirm the details before submission.
Design the Drain Management architecture	Designed	Figure 4.1 shows the two units and their API routes.
Measure water level and rainfall using the main ESP32 unit	Implemented in supplied firmware	Confirm the reference height and record actual sensor tests.

Compare three points to identify possible blockages	Implemented in supplied firmware	Validate the 3.0 cm threshold through normal-flow and blockage tests.
Require three consecutive abnormal readings	Implemented in supplied firmware	Record how many alerts are correctly triggered and how long confirmation takes.
Send readings to the backend	Firmware configured	Verify HTTP success codes and the saved records for both routes.
Connect records to Flood Guard 360 dashboard/map	Integration target described	Add evidence from a successful end-to-end test; confirm location mapping.
Calculate numerical sensor and detection performance	Not evidenced in supplied files	Add the actual test log and calculate the metrics in Section 3.4.
Implement riverbank-gap flood warning	Future extension	Add the rule to firmware and test before claiming completion.

8.3 Individual research and product contributions

My individual contribution was to develop the IoT hardware and ESP32 firmware for real time drain water level and rainfall monitoring, implement multi point blockage detection, and prepare the data flow to the backend through API endpoints. I also worked on the sensor configuration, reference height settings, threshold logic, and controlled prototype testing.

The main technical contribution is the comparison of three water level measurements and the use of consecutive abnormal readings to identify a possible blockage section. The main product contribution is a two unit prototype that can provide both local monitoring readings and structured data for the wider Flood Guard 360 platform.

8.4 Future directions

The next step is to complete repeated tests and record the actual sensor readings, false alerts, detection outcomes, and HTTP results. The sensor reference heights should be measured again, and the main unit setting of 700 cm should be corrected if it does not match the real setup. The rain sensor should also be calibrated with a known procedure if the project needs a rainfall value rather than a relative wetness percentage.

The blockage method can be improved by collecting normal flow measurements at each site and choosing a threshold based on the usual difference between points. Additional sensors could cover a longer drain. A better test channel and a controlled obstruction could make the experiments closer to real conditions. Long term outdoor tests would be required to study dirt, rain, waves, sensor movement, and network failures.

The bank gap flood warning can be implemented as a separate rule by measuring the gap between the water surface and the riverbank, setting a site-specific warning threshold, and testing it at different water levels. The backend can also be improved by validating input data, using secure communication, adding server timestamps, and storing sensor health information.

After a reliable history of water level and rainfall readings has been collected, a machine learning model may be trained to estimate future water levels. The model should be compared with a simple baseline and tested on data it has not seen during training. Finally, the Drain Management component can be fully linked with the Flood Mapping component so that stored readings are connected to the correct map location and presented with clear warning labels.

REFERENCES

[1] A. Rajasekar, M. Suman, J. S. Jai Aditya, S. Danush, and K. Shyam Sundar, "Drain block detection and controlling system," in *Proc. 2024 Int. Conf. Commun., Comput. Internet of Things (IC3IoT)*, 2024, doi: 10.1109/IC3IoT60841.2024.10550345.

[2] V. S. Gawali, M. Pande, M. Sayyad, and R. S. Bhadade, "IoT based real time drainage monitoring system," in *Proc. 2024 Int. Conf. Emerging Smart Comput. Informat. (ESCI)*, Pune, India, 2024, pp. 1–6, doi: 10.1109/ESCI59607.2024.10497198.

[3] T. Jayapriya, L. Bhuvaneswari, A. Asmitha, S. Durka, K. Suman, and R. Gokila, "IoT based smart drainage monitoring and alert system using mobile application integration," in *Proc. 2025 4th Int. Conf. Innovative Mechanisms Ind. Appl. (ICIMIA)*, 2025, pp. 227–232, doi: 10.1109/ICIMIA67127.2025.11200687.

[4] S. Chakraborty, S. Chatterjee, A. Sarkar, S. Kanjilal, G. Das, and P. Kundu, "Study of IoT enabled water logging detection and control in drainage system for smart city," in *Proc. 2024 Int. Conf. Big Data Analytics Bioinformat. (DABCon)*, 2024, doi: 10.1109/DABCON63472.2024.10919387.

[5] M. Z. S. Hadi, V. R. Putri, and M. Yuliana, "Design of water level and drainage monitoring system based on Internet of Things," in *Proc. 2023 IEEE Int. Conf. Commun., Netw. Satell. (COMNETSAT)*, Malang, Indonesia, 2023, pp. 570–576, doi: 10.1109/COMNETSAT59769.2023.10420559.

[6] A. Diriyana, U. Darusalam, and N. D. Natasha, "Water level monitoring and flood early warning using microcontroller with IoT based ultrasonic sensor," *Jurnal Teknik Informatika C.I.T*, vol. 11, no. 1, pp. 22–28, 2019.

[7] J. G. Natividad and J. M. Mendez, "Flood monitoring and early warning system using ultrasonic sensor," *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 325, Art. no. 012020, 2018, doi: 10.1088/1757-899X/325/1/012020.

[8] W. Indrasari, B. H. Iswanto, and M. Andayani, "Early warning system of flood disaster based on ultrasonic sensors and wireless technology," *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 335, Art. no. 012005, 2018, doi: 10.1088/1757-899X/335/1/012005.

[9] S. Dswilan, Harmadi, and Marzuki, "Flood monitoring system using ultrasonic sensor SN-SR04T and SIM 900A," *J. Phys.: Conf. Ser.*, vol. 1876, Art. no. 012003, 2021, doi: 10.1088/1742-6596/1876/1/012003.

[10] B. E. Pengel *et al.*, "Flood early warning system: sensors and internet," in *Floods: From Risk to Opportunity*, A. Chavoshian and K. Takeuchi, Eds., IAHS Publ. 357. IAHS Press, 2013, pp. 445–453.

[11] K. I. Dissanayake, K. D. J. Fernando, C. Weerarathne, D. Kavinda, and H. G. B. Chathuranga, "Design and experimental evaluation of an IoT-enabled semi-autonomous platform for sediment level detection and 3D mapping in urban canals," in *Proc. 2026 Moratuwa Eng. Res. Conf. (MERCon)*, 2026, doi: 10.1109/MERCon71835.2026.11691390.

[12] S. P. M. K. W. Ilukkumbure, V. Y. Samarasiri, M. F. Mohamed, V. Selvaratnam, and U. U. S. Rajapaksha, "Early warning for pre and post flood risk management by using IoT and machine learning," in *Proc. 2021 3rd Int. Conf. Adv. Comput. (ICAC)*, 2021, pp. 252–257, doi: 10.1109/ICAC54203.2021.9671141.

[13] Z. Farzan, "833,000 affected, death toll rises to 159 in Sri Lanka's worst floods in decades," *News 1st*, Nov. 30, 2025. [Online]. Available: https://newsfirst.lk/2025/11/30/833-000-affected-death-toll-rises-to-159-in-sri-lanka-s-worst-floods-in-decades [Accessed: Oct. 7, 2026].

[14] "Sri Lanka: Over 120,000 affected by severe flooding in west," *IRIN / The New Humanitarian*, May 2007. [Online]. Available: https://www.thenewhumanitarian.org/news/2007/05/07 [Accessed: Oct. 7, 2026].

[15] "Development of an integrated urban drainage cycle management model for city of Colombo," in *Proc. KDU Int. Res. Conf.*, 2014. [Online]. Available: https://ir.kdu.ac.lk/handle/345/1529 [Accessed: Oct. 7, 2026].

[16] A. Abeysinghe, "Flooding in Colombo aggravated by human activity," *AsiaNews*, Sep. 13, 2023. [Online]. Available: https://www.asianews.it/en/south-asia/sri-lanka/flooding-in-colombo-aggravated-by-human-activity [Accessed: Oct. 7, 2026].

GLOSSARY

Freeboard: The vertical gap between the water surface and the top of a bank or channel edge.

HTTP response: The message returned by a server after it receives a request.

Possible blockage: A status raised when neighboring water levels differ beyond the configured threshold for the required number of readings.

Reference height: The measured height used to convert sensor to water distance into water depth.

Threshold: A chosen value used to decide when a measurement should be treated as abnormal.

Upstream and downstream: Locations before and after the direction of water flow, respectively.

APPENDICES

Appendix A Research instruments or extended results

Table 15 : Prototype settings and illustrative worked example

Item	Filled value / result
Main-unit sampling interval	20 seconds, according to the supplied ESP32-WROOM-32U firmware.
Main-unit ultrasonic sensor	TRIG: GPIO 27; ECHO: GPIO 14. The firmware reference height is 700 cm; confirm this against the actual installation before relying on the calculated water level.
Rain sensor settings	Digital output: GPIO 4; analog output: GPIO 34; ADC resolution: 12-bit. Dry calibration value: 4095; wet calibration value: 2595. The output is relatively wet, not rainfall in millimeters.
Main-unit API route	HTTP POST to port 3001 at /api/sensor-readings/ingest.
Blockage sensor pin mapping	S1 (upstream): TRIG GPIO 5, ECHO GPIO 6. S2 (middle): TRIG GPIO 7, ECHO GPIO 15. S3 (downstream): TRIG GPIO 16, ECHO GPIO 17.
Blockage sensor mounting heights in firmware	S1: 21.00 cm; S2: 21.49 cm; S3: 19.98 cm.
Blockage sampling interval	10 seconds.
Blockage decision settings	Neighboring water-level difference threshold: 3.00 cm. Required consecutive abnormal readings: 3. This gives a minimum confirmation period of about 30 seconds when readings are taken on schedule.
Blockage API route	HTTP POST to port 3002 at /api/blockage-readings/ingest.
Illustrative sensor	S1: 10.00 cm; S2: 14.00 cm; S3: 12.00 cm.

distances	
Illustrative calculated water levels	Water level = mounting height − measured distance. S1: 21.00 − 10.00 = 11.00 cm. S2: 21.49 − 14.00 = 7.49 cm. S3: 19.98 − 12.00 = 7.98 cm.
Illustrative neighboring differences	S1–S2: |11.00 − 7.49| = 3.51 cm. S2–S3: |7.49 − 7.98| = 0.49 cm.

Appendix B Academic integrity and AI use disclosure

Academic Integrity Declaration

I declare that this individual thesis, titled *Real-Time Drain Water Level Monitoring, Rainfall Monitoring, and Multi-Point Blockage Detection for Flood Risk Management*, has been prepared in accordance with the principles of academic integrity. All external sources, research papers, datasets, software tools, and other materials used in this thesis must be properly acknowledged and referenced.

I acknowledge that I am responsible for the accuracy, originality, technical correctness, and proper reference of this thesis. I understand that the research findings, test results, technical details, and implementation descriptions must accurately represent the work carried out during this research.

AI Use Disclosure

Generative AI tools were used to assist with drafting, organizing, and improving selected sections of this thesis. This assistance included simplifying technical explanations, improving grammar and sentence structure, organizing thesis chapters, and preparing descriptions of the Drain Management component of the Flood Guard 360 project.

The project details, individual component, hardware setup, software implementation, and research activities were based on the project materials and information provided by the student. AI-generated content was reviewed for relevance and consistency

with the available project information. The student remains responsible for checking the accuracy of technical descriptions, research references, test results, implementation details, and conclusions before submission.

AI assistance was not used as a replacement for hardware development, software implementation, experimental testing, independent analysis, or the verification of academic references. Any code, measurements, test results, and conclusions included in the final thesis must accurately reflect the work carried out by the student.

AI Tools Used and Their Purposes

Tool: ChatGPT (OpenAI)

Purpose of use:

- Assisting with the organization and drafting of thesis chapters and subsections.
- Improving the clarity, grammar, sentence structure, and academic writing style of the thesis.
- Preparing explanations of the Drain Management component, including real-time water-level monitoring, rainfall monitoring, and multi-point blockage detection.